Aula 11

Testes a partir dos critérios de aceite

Unidade 3 — Construção

Nível de IA: IA como colaboradora

De onde viemos

Unidade 3 · Aula 10 entregue em 29/10

- A história foi cortada em fatias, cada uma com o seu critério — hoje eles viram testes.
- A definição de pronto exige teste com o nome do critério; hoje o teste passa a ser o centro.
- O PostgreSQL está no ar; a troca do código é na Aula 12 — e depende da suíte de hoje.

Onde estamos

O produto cresce — e cada mudança pode quebrar o que já funcionava

- Quanto mais histórias, mais chances de uma quebrar a outra sem ninguém ver.
- Na Aula 12 o banco muda por baixo do código: sem teste, é aposta.
- Hoje: critérios viram testes, testes viram rede, e a rede é posta à prova.

Objetivo da aula

Transformar os critérios de aceite em testes que falham quando a regra falha — e que protegem a troca de banco da próxima aula.

Evidência que sai desta aula: um PR de testes de uma história: os critérios, dois casos limite que o critério não previa e o registro da inversão de uma regra.

Mapa da aula: 1 critério → teste · produção A → 2 casos limite → 3 rede de proteção → 4 IA gerando testes · produção B. O exemplo que atravessa os quatro: o doador cancela uma doação.

1 Do critério de aceite ao teste

O critério é o acordo em linguagem de negócio; o teste é a mesma frase escrita para a máquina repetir. O Dado vira a preparação do estado, o Quando vira a ação e o Então vira a verificação. Na API do Prato Cheio, o teste mais útil passa pela rota, a regra e o SQL — é de integração; uma regra pura, sem banco, seria de unidade.

```
Critério: dado que o doador publicou uma doação,
          quando ele a cancela,
          então ela não aparece na lista de disponíveis.

Teste:    publicar → cancelar(id) → GET /api/doacoes → []
```

Três frases, três passos

DO CRITÉRIO DE ACEITE AO TESTE · O CANCELAMENTO

1 Dado

NO CRITÉRIO

que o doador publicou uma doação

NO TESTE: PREPARAR O ESTADO

```
POST /api/doacoes
```

›

2 Quando

NO CRITÉRIO

ele a cancela

NO TESTE: PROVOCAR A AÇÃO

```
POST
/api/doacoes/:id/cancelar
```

›

3 Então

NO CRITÉRIO

ela não aparece na lista de disponíveis

NO TESTE: VERIFICAR O RESULTADO

```
GET /api/doacoes
expect(res.body).toHaveLength(0)
```

O NOME DO TESTE É A REGRA

"teste 1"	→	"remove da lista a doação cancelada"
"deve funcionar"	→	"recusa cancelar uma doação já aceita"

o teste um critério, um teste, **um nome que diz a regra** — é ele que aparece quando o CI fica vermelho.

1 No Prato Cheio · o critério da fatia 1 vira teste

O CRITÉRIO (CA1 DO CANCELAMENTO)

Dado que o doador publicou uma doação, quando ele a cancela, então ela não aparece na lista de disponíveis.

O TESTE, EM TESTS/DOACOES.TEST.JS

```
it('remove da lista a doação cancelada', async () => {
// Dado: uma doação publicada
const { body } = await request(app)
  .post('/api/doacoes').send(doacao);
// Quando: o doador a cancela
await request(app)
  .post(`/api/doacoes/${body.id}/cancelar`);
// Então: ela não aparece na lista
const res = await request(app).get('/api/doacoes');
expect(res.body).toHaveLength(0);
});
```

POR QUE ESTE TESTE SERVE

- **Dado** pela própria API: não depende de dado que já estava lá
- **Uma ação** no Quando; o resto é preparação e verificação
- **Confere o valor** da lista, não só o status 200

QUE TIPO DE TESTE É

Passa por **rota, regra e SQL**: é de **integração**. Uma regra pura, sem banco, seria de **unidade**. Para o critério, o de integração é o que importa.

a leitura sublinhe no teste o Dado, o Quando e o Então: **se algum não aparece, falta algo.**

1 Do critério ao teste

7 min

ATIVIDADE · em duplas, depois em voz alta

Critério da fatia 1 da coleta: "dado que a ONG aceitou a doação, quando o voluntário registra a coleta, então a doação aparece como coletada". Escrevam o teste.

1. Dado: que chamadas preparam o estado?
2. Quando: qual é a chamada que provoca a ação?
3. Então: o que se verifica — e em que lugar da resposta?
4. Nome: uma frase que diga a regra, sem a palavra "teste".

PARA IR ALÉM

Do critério ao teste

Escrever o critério como Dado, Quando, Então e transformá-lo em teste automatizado tem uma linhagem conhecida.

Referência	Ano	O que você tira dela
NORTH, D. Introducing BDD (dannorth.net)	2006	a forma Dado–Quando–Então para escrever critérios
BECK, K. Test-Driven Development: By Example	2002	escrever o teste primeiro e vê-lo falhar
FOWLER, M. TestPyramid (martinfowler.com)	2012	unidade, integração e interface: quanto de cada
COHN, M. Succeeding with Agile	2009	a origem da pirâmide de testes

Leitura opcional. O texto de North é curto e de acesso livre; comece por ele.

Referências · não substituem a sua consulta

2 Casos limite e caminhos de erro

O caminho feliz mostra que funciona; o limite mostra o que acontece quando não funciona. É nos limites que o produto quebra para o usuário, e quase nunca estão no critério. Quatro perguntas servem para qualquer um: e se faltar algo, e se vier vazio, e se repetir, e se não existir.

Situação	O que deveria acontecer	Testado?
Cancelar com um id que não é número	recusar com mensagem clara	geralmente não
Cancelar a mesma doação duas vezes	recusar a segunda	às vezes
Cancelar uma doação que não existe	recusar com mensagem clara	quase nunca

Quatro perguntas que revelam o limite

QUATRO PERGUNTAS QUE REVELAM O CASO LIMITE

C `o doador cancela uma doação disponível`

o critério só prevê o caminho feliz →

1 E se faltar algo?

NO CANCELAMENTO

cancelar com um id que não é número, como `/doacoes/abc/cancelar`

O TESTE ESPERA

recusa, com mensagem clara

2 E se vier vazio?

NO CANCELAMENTO

cancelar a única doação e consultar a lista

O TESTE ESPERA

lista vazia, e não um erro

3 E se repetir?

NO CANCELAMENTO

cancelar a mesma doação duas vezes

O TESTE ESPERA

a segunda tentativa é recusada

4 E se não existir?

NO CANCELAMENTO

cancelar um id que ninguém publicou

O TESTE ESPERA

recusa, sem derrubar o servidor

a regra para cada critério, **pelo menos um teste do que não deve acontecer** — hoje, dois que o critério não previa.

2 No Prato Cheio · um limite que o critério não previa

UM CASO QUE O CRITÉRIO NÃO PREVIA: CANCELAR UM ID QUE NÃO EXISTE

1 Pergunte

e se o doador cancelar uma doação que **ninguém publicou**?

UMA DAS QUATRO PERGUNTAS

"E se não existir?"

2 Decida o esperado

o produto **recusa**, com mensagem clara, e segue de pé.

COMO A API RESPONDE

```
400
{ erro: "doação
não encontrada"
}
```

3 Escreva o teste

```
it("recusa cancelar uma
  doação que não existe",
  async () => {
  const res = await request(app)
    .post("/api/doacoes/9999/cancelar");
  expect(res.status).toBe(400);
  expect(res.body.erro)
    .toMatch(/não encontrada/);
});
```

4 Rode e leia

```
FAIL recusa
cancelar uma doação
que não existe
expected 200 to be
400
```

O QUE O VERMELHO MOSTROU

um **buraco no produto**: cancelar o que não existe respondia 200.

DEPOIS DO CONSERTO

verde — e a regra passou a ter guarda.

OS OUTROS LIMITES — o mesmo ciclo vale para **id que não é número · cancelar duas vezes · cancelar a única doação** e consultar a lista

o ciclo — perguntar, decidir, escrever, rodar: **vermelho e verde informam** — o vermelho, antes de o usuário descobrir.

2 Quatro perguntas, um limite

6 min

ATIVIDADE · em trios, depois em voz alta

Critério: "a ONG vê as doações que aceitou". Apliquem as quatro perguntas e, para dois casos, escrevam o nome do teste e o que ele espera.

1. E se faltar algo? A consulta chega sem dizer qual é a ONG.
2. E se vier vazio? A ONG ainda não aceitou nenhuma doação.
3. E se repetir? A ONG aceita a mesma doação duas vezes.
4. E se não existir? A consulta é de uma ONG que ninguém cadastrou.

Faltou · vazio · repetiu · não existe · gabarito nas notas do slide

PARA IR ALÉM

Casos limite e testes de erro

Valor limite e caminho de erro são técnicas antigas de teste — e o que mais falta nos testes escritos às pressas.

Referência	Ano	O que você tira dela
MYERS, G. The Art of Software Testing	1979	partição de equivalência e análise de valor limite
KANER, C.; FALK, J.; NGUYEN, H. Testing Computer Software	1999	como pensar os casos de erro de um programa
MARTIN, R. C. Clean Code (cap. 17, T5)	2008	"teste as condições de contorno"

Leitura opcional. Myers é o clássico: o capítulo sobre projeto de casos de teste basta.

Referências · não substituem a sua consulta

3 Teste como evidência e como rede de proteção

Um teste que passa é evidência objetiva de que o comportamento existe; o conjunto é a rede que deixa mudar o código sem apostar. Refatorar é mudar a estrutura sem mudar o comportamento, e só os testes provam que o comportamento ficou. Na próxima aula o banco inteiro é trocado — e quem diz se algo mudou é a suíte de hoje.

Trocar SQLite por PostgreSQL sem testes

Trocar com os mesmos testes verdes antes e depois

Verde, mexe, verde

A REDE DE PROTEÇÃO: VERDE, MEXE, VERDE

1 Verde antes

Rode `npm test` antes de tocar em qualquer coisa.

POR QUÊ

sem o verde de partida, você não sabe se o vermelho é seu.

›

2 Mude só a estrutura

Renomear, extrair, trocar a implementação.

A REGRA

o comportamento fica igual. Se mudar, é outra mudança.

›

3 Verde depois

Rode a suíte de novo.

SE FICOU VERMELHO

o nome do teste diz **qual comportamento** mudou.

O QUE A REDE SEGURA

O que um teste observa

- a **resposta da API**: status e corpo
- a **regra** de cada critério de aceite
- o **estado depois** de cada ação

ONDE A REDE TEM BURACO

O que ninguém avisa

- **critério sem teste**: a mudança passa em silêncio
- **teste decorativo**: passa com a regra errada
- comportamento que **nenhum teste exercita**

a evidência na Aula 12 a suíte de hoje **é a prova** de que a troca de banco não mudou o comportamento.

3 No Prato Cheio · o que cada teste protege na Aula 12

MUDANÇA DA AULA 12	O QUE PODE QUEBRAR	TESTE DE HOJE QUE AVISA
Trocar SQLite por PostgreSQL: os marcadores `?` viram `$1`	o **UPDATE** do cancelar pega a doação errada, ou nenhuma	**"remove da lista a doação cancelada"** CA1 · o teste do critério
Trocar o tipo da data: de texto para `DATE`	a **data do cancelamento** volta em outro formato	**"registra a data do cancelamento"** CA3 · o teste do critério
Trocar o tipo do id: de inteiro para texto ou serial	um id inválido passa a **estourar** em vez de ser recusado	**"recusa cancelar com id inválido"** caso limite
Renomear `d` para `doacao`; extrair uma função	**nada** deveria mudar para quem usa	**a suíte inteira continua verde** a prova de que nada mudou

BURACO comportamento **sem teste** é comportamento que a mudança altera em silêncio — ninguém fica vermelho para avisar.

a rede antes de mexer: **verde.** Depois de mexer: **verde.** Se ficou vermelho, o nome do teste diz o que mudou.

3 O que esta suíte deixa passar

6 min

ATIVIDADE · em duplas, depois em voz alta

A suíte tem quatro testes: mostra a doação publicada na lista · remove da lista a doação cancelada · recusa cancelar doação já aceita · recusa publicar doação sem campos obrigatórios. Para cada mudança da Aula 12, qual teste avisa — ou nenhum?

1. Trocar os marcadores ? por $1 em repositorio.js.
2. Mudar a data de validade de texto para DATE.
3. Extrair a validação de campos obrigatórios para uma função.
4. Trocar o id de inteiro para texto.

Qual teste avisa? Ou nenhum? · gabarito nas notas do slide

PARA IR ALÉM

Refatorar com testes

Refatorar sem testes é só mudar o código e torcer. A literatura sobre refatoração começa pela suíte.

Referência	Ano	O que você tira dela
FOWLER, M. Refactoring (2ª ed.)	2018	refatorar em passos pequenos, com testes autoverificáveis
FEATHERS, M. Working Effectively with Legacy Code	2004	código sem teste é código legado
FOWLER, M. SelfTestingCode (martinfowler.com)	2014	o produto que se verifica sozinho

Leitura opcional. O texto de Fowler sobre código autoverificável tem uma página.

Referências · não substituem a sua consulta

4 Usar IA para gerar testes: cobertura real × aparente

A IA gera muitos testes, e todos passam. Passar não é o objetivo: o objetivo é falhar quando o código está errado. Teste que passa com a regra errada é cobertura aparente — dá confiança sem dar proteção. A verificação é uma só: inverter a regra e contar quantos testes ficam vermelhos.

```
// gerado por IA — parece bom, não verifica nada
it('cancela uma doação', async () => {
  const res = await cancelar(1);
  expect(res).toBeDefined();   // sempre passa
});
```

Quem testa o teste: inverter uma regra

QUEM TESTA O TESTE: INVERTER UMA REGRA

1 Escolha uma regra

uma regra de um critério, **com teste**.

```
doação já aceita não pode ser cancelada
```

›

2 Inverta no código

quebre a regra de propósito em `src/doacoes.js`.

```
apague o if (status === "aceita")
de cancelar
```

›

3 Rode a suíte

rode tudo e **não corrija nada** ainda.

```
$ npm test
```

›

4 Conte os vermelhos

leia a última linha: quantos **perceberam**?

```
Tests 1 failed | 7 passed
```

0 VERMELHOS

Suíte decorativa

todos passaram com a regra errada: **nenhum teste guarda essa regra**. Falta teste, ou ele só confere status e formato.

1 OU MAIS VERMELHOS

A regra tem guarda

o nome do teste vermelho diz **qual regra caiu**. É o comportamento que dá para refatorar com segurança.

AO FINAL

Desfaça

```
git restore src/doacoes.js
```

commite antes de inverter. A inversão é só um experimento e **não entra no PR**.

a entrega o número de vermelhos é a evidência: **quantos testes deixaram de pegar o erro**.

4 No Prato Cheio · testes gerados por IA, postos à prova

PEDIDO À IA `"escreva os testes do cancelamento de doação"` — três testes, todos verdes

1 · O QUE A IA DEVOLVEU

```
it('cancela uma doação', async () => {
  const res = await cancelar(1);
  expect(res).toBeDefined(); // sempre passa
});
it('lista depois de cancelar', async () => {
  const res = await get('/api/doacoes');
  expect(res.status).toBe(200); // só a rota
});
it('não cancela doação aceita', async () => {
  const res = await cancelar(1);
  expect(res.status).toBeLessThan(500);
});
```

2 · INVERTEMOS A REGRA

apagamos o `if` que recusa cancelar doação aceita e rodamos:

```
✓ cancela uma doação
✓ lista depois de cancelar
✓ não cancela doação aceita

Tests 0 failed | 3 passed
```

Cobertura aparente: a regra caiu e ninguém notou.

3 · REESCREVEMOS O TERCEIRO

```
it('recusa cancelar uma doação já aceita',
  async () => {
  // Dado: publicada e aceita
  const id = await publicarEAceitar();
  const res = await cancelar(id);
  expect(res.status).toBe(400);
  expect(res.body.erro)
    .toMatch(/já aceitou/);
});
```

Agora, **sem o if, ele fica vermelho: cobertura real.**

a pergunta para cada **`expect` o que precisa quebrar para ele falhar?**

4 Este teste falharia?

7 min

ATIVIDADE · em duplas, depois em voz alta

Uma IA devolveu estes expect para o teste de registrar a coleta. Para cada um, digam o que precisa quebrar no código para ele ficar vermelho e classifiquem: decorativo, só a rota, confere o dado ou guarda a regra.

1. expect(res.body).not.toBeNull()
2. expect(res.status).toBe(200)
3. expect(typeof res.body.id).toBe('number')
4. expect(res.body.status).toBe('coletada')
5. coletar uma doação que ninguém aceitou → expect(res.status).toBe(400)

O que precisa quebrar para ele falhar? · gabarito nas notas do slide

PARA IR ALÉM

Cobertura, mutação e testes gerados por IA

Cobertura alta não prova que os testes servem; a pergunta é se eles falham quando o código está errado.

Referência	Ano	O que você tira dela
INOZEMTSEVA, L.; HOLMES, R. Coverage Is Not Strongly Correlated with Test Suite Effectiveness (ICSE)	2014	cobertura de linhas não mede se o teste pega o erro
JIA, Y.; HARMAN, M. An Analysis and Survey of the Development of Mutation Testing (IEEE TSE)	2011	inverter o código para ver se os testes percebem
SCHÄFER, M. et al. An Empirical Evaluation of Using Large Language Models for Automated Unit Test Generation (IEEE TSE)	2024	testes gerados por modelo: o que passa e o que presta
FOWLER, M. TestCoverage (martinfowler.com)	2012	cobertura serve para achar o que falta, não para medir qualidade

Leitura opcional. O texto de Fowler sobre cobertura é curto e de acesso livre.

O que entregar hoje

Uma entrega de 0,25 — cada integrante entrega no Teams até 23h59 de hoje

- Um PR de testes de uma história — a da Aula 10, de preferência, ou a dos it.todo do template —, com 3 ou mais testes de critério: um it com o nome do critério, não it.todo.
- 2 ou mais casos limite ou de erro que o critério não previa, cada um com o nome da regra.
- Na descrição do PR, a seção ## Inversão de regra: qual regra, o que foi invertido, que testes ficaram vermelhos — e que a inversão foi desfeita.
- Uso de IA preenchido, CI verde no PR e um commit seu, de hoje — ou Co-authored-by: no commit do par.
- Fora da nota, mas necessário para a Aula 12: a suíte verde na main e o PostgreSQL respondendo.

Entrega do encontro: o PR de testes (aberto ou integrado), com a seção Inversão de regra

Detalhes e correção: enunciado-atividade-ava.md · correção binária: entregou no prazo e no formato, pontuou

Produção em sala, em duas faixas

Em grupo (3 a 5 pessoas) — no repositório do grupo, nesta ordem

1. A1 · Escolher a história e listar os critérios; trocar os it.todo, ou criar os testes, um it com o nome de cada critério.
2. A2 · Fazer as quatro perguntas a cada critério e escrever 2 testes de limite ou de erro. Se um ficar vermelho, é um buraco do produto: corrigir em commit à parte.
3. A3 · Rodar npm test, deixar o CI verde e abrir o PR de testes, com a descrição já iniciada.
4. B1 · Escolher uma regra com teste, commitar, inverter no código, rodar a suíte e contar os vermelhos. Com zero vermelhos, reforçar o teste e repetir.
5. B2 · Desfazer a inversão, escrever a seção ## Inversão de regra e o Uso de IA; outro integrante revisa; CI verde; merge se der tempo.

Faixa A: 19h50–20h40, depois dos pontos 1 e 2 · Faixa B: 21h12–22h20, depois dos pontos 3 e 4

Nível de IA nesta aula

IA como colaboradora

Pode gerar testes. Em cada expect, pergunte: o que precisa quebrar para ele falhar? Inverta uma regra para conferir.

Em qualquer nível, você continua responsável por verificar, testar, corrigir e defender o resultado.

Sua consulta

O que você precisa ter à mão para escrever um teste que diga o critério, achar o limite que ninguém previu e reconhecer, em cinco segundos, o teste que passa sem verificar nada?

Manuscrita, de próprio punho, extensão a seu critério. É a única consulta permitida na prova.

Para estudar · Mapa da aula

MAPA DA AULA · O FIO: O DOADOR CANCELA UMA DOAÇÃO

1 Do critério ao teste

PERGUNTA QUE RESPONDE
Como a frase de negócio vira uma verificação que a máquina repete?

NO REPOSITÓRIO
um `it` com o nome do critério, no lugar do `it.todo`

EXERCÍCIO
C1 · o critério da coleta vira teste (7 min)

›

2 Casos limite e erro

PERGUNTA QUE RESPONDE
O que acontece quando o caminho feliz não acontece?

NO REPOSITÓRIO
dois testes que o critério não previa, nomeados pela regra

EXERCÍCIO
C2 · quatro perguntas sobre um critério (6 min)

›

3 Teste como rede

PERGUNTA QUE RESPONDE
O que me deixa mexer no código na Aula 12 sem apostar?

NO REPOSITÓRIO
a suíte verde, antes e depois de cada mudança

EXERCÍCIO
C3 · o que esta suíte deixa passar (6 min)

›

4 IA gerando testes

PERGUNTA QUE RESPONDE
Este teste falharia se o código estivesse errado?

NO REPOSITÓRIO
a inversão de uma regra, com o número de vermelhos

EXERCÍCIO
C4 · este teste falharia? (7 min)

ENTREGA DE HOJE — um **PR de testes** da história: critérios, dois limites e o **registro da inversão** · CI verde

na Aula 12 — os testes de hoje **são a prova** de que a troca de banco não mudou o comportamento.

Para estudar · Glossário

Termo	Em uma frase
Critério de aceite	o acordo, em linguagem de negócio, do que a história faz
Teste de critério	o mesmo acordo escrito para a máquina: Dado, Quando, Então
Teste de unidade	confere uma regra isolada, sem banco nem rota
Teste de integração	passa por rota, regra e SQL — o mais útil para o critério
Caso limite	o que acontece nas bordas: faltou, veio vazio, repetiu, não existe
Cobertura aparente	testes que passam, mas passariam com a regra errada
Cobertura real	testes que ficam vermelhos quando a regra falha
Inversão de regra	quebrar a regra de propósito para ver quantos testes percebem
Rede de proteção	suíte verde antes e depois de mudar a estrutura

Cobertura de linhas e cobertura real são coisas diferentes: a primeira conta o que rodou, a segunda o que seria percebido.

Para estudar · Um PR de testes completo

```
#18  Testes do cancelamento de doação
## O que este PR faz      testes de CA1 a CA3 e dois casos limite
## História / decisão     "Doador cancela" · CA1, CA2, CA3
## Como verificar         npm test: 5 testes novos, todos verdes
## Inversão de regra      regra: doação aceita não se cancela (doacoes.js)
                          removido o if → 1 vermelho: "recusa cancelar uma
                          doação já aceita" · inversão desfeita
## Checklist              [x] CI verde  [x] aprovado por outro integrante
## Uso de IA              rascunho dos testes; 2 expect decorativos trocados
```

Exemplo ilustrativo: o PR de testes do cancelamento. O seu segue o mesmo modelo, em .github/pull_request_template.md, com a seção Inversão de regra acrescentada.

Para estudar · O que levar

Uma frase por ponto — para escrever na sua consulta com as suas palavras

1. O teste diz o critério: o Dado prepara, o Quando age, o Então verifica; o nome é a regra.
2. O caminho feliz prova que funciona; o limite prova o que acontece quando não funciona.
3. Teste é evidência e rede: verde antes, mexe só na estrutura, verde depois.
4. A IA gera testes que passam; o que vale é o teste que falha com a regra errada.
5. Na dúvida, inverta a regra e conte os vermelhos: zero é suíte decorativa.

O deck não substitui a sua consulta: ela é manuscrita e feita por você.

Para estudar · Autoteste

Responda por escrito e depois confira nas notas do slide

1. Por que expect(res).toBeDefined() não protege nada?
2. Escreva o nome de um teste para "a ONG não vê doação cancelada".
3. Aplique as quatro perguntas do limite ao critério "o voluntário registra a coleta".
4. Você inverteu uma regra e nenhum teste ficou vermelho. O que isso diz, e o que você faz?
5. Na Aula 12 um teste ficou vermelho depois da troca de banco. É bom ou ruim?

Se travar em alguma, volte ao infográfico do ponto correspondente.